P1 Report

by

Sahil Ishan Tonmoy
22301612
S.M. Mashrafi Zaman
22301614
Shafayet Karim
22101507
Shifat Ahmed
22301582

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
October 2025.

© 2024. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Sahil Ishan Tonmoy	S.M. Mashrafi Zaman
22301612	22301614
Shafayet Karim	Shifat Ahmed
22101507	22301582

Approval

The thesis/project titled "Adaptive Threat Detection for Securing Cloud CI/CD Pipelines Using Evolving Neural Architectures" submitted by

1. Sahil Ishan Tonmoy (22301612)
2. S.M. Mashrafi Zaman (22301614)
3. Shafayet Karim (22101507)
4. Shifat Ahmed (22301582)

of [Summer], [2025] has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in [Summer] Year.

Examining Committee:

Supervisor:
(Member)

Mr. Partha Bhoumik

Lecturer
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)

Name of Co-Supervisor

Designation
Department
Institution

Thesis Coordinator:
(Member)

Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Ethics Statement (Optional)

This is optional, if you don't have an ethics statement then omit this page

Abstract

Cloud computing pipelines, particularly Continuous Integration and Continuous Deployment (CI/CD) environments, are essential for modern software development due to their flexibility, scalability, and automation. However, these dynamic cloud infrastructures face continuously evolving cyber threats, including polymorphic and metamorphic malware, which traditional static detection systems struggle to handle. Existing systems rely on fixed architectures, require new threat samples for retraining, and lack self-learning capabilities in sync with the pipeline, often resulting in high false positives, operational delays, and potential data breaches or financial loss. To address these limitations, this research proposes an adaptive malware detection system for cloud CI/CD pipelines. The system uses deep neural networks to perform both static and behavioral analysis of pipeline artifacts such as code, build logs, and containers. A key feature is its self-evolving capability, where the neural network can restructure and adjust its parameters dynamically in response to new behaviors, enabling the system to adapt to previously unseen threats. Unlike post-deployment scanners, this model is embedded directly in the pipeline, allowing real-time monitoring and detection. Additionally, the system includes a threat mutation module that generates modified versions of known malware and injects them into the detection environment. This allows the model to learn from controlled adversarial samples while distinguishing them from external threats, thereby improving detection accuracy and reducing false positives. By continuously learning from both normal operational patterns and self-generated threats, the system maintains resilience against evolving malware and strengthens the security of cloud pipelines. The expected contribution of this research is a self-adaptive neural framework capable of evolving with changing threat behavior and providing proactive, context-aware protection for cloud CI/CD pipelines, addressing the shortcomings of current static detection systems, and enhancing the reliability, efficiency, and security of modern cloud software delivery.

Keywords: Adaptive Evolving Neural Security Architectures, Cloud-Native CI/CD Security and Real-Time Defense, Polymorphic and Metamorphic Malware Detection, Behavioral and Code-Level Anomaly Detection, Static and Dynamic Threat Analysis (SAST/DAST), Adversarial Training, Red Team Simulation

Dedication (Optional)

A dedication is the expression of friendly connection or thanks by the author towards another person. It can occupy one or multiple lines depending on its importance. You can remove this page if you want.

Acknowledgement

Table of Contents

Declaration i

Approval ii

Ethics Statement iv

Abstract v

Dedication vi

Acknowledgment vii

Table of Contents viii

List of Figures x

List of Tables xi

Nomenclature xi

1 Introduction 1
- 1.1 Background . . . 2
- 1.2 Rational of the Study or Motivation . . . 2
- 1.3 Problem Statement . . . 3
- 1.4 Objectives . . . 3
- 1.5 Methodology . . . 4
- 1.6 Scope and Limitations . . . 5
  - 1.6.1 Scope of the Research . . . 5
  - 1.6.2 Research Challenges . . . 5

2 Literature Review 6
- 2.1 Preliminaries . . . 6
  - 2.1.1 Adaptive Evolving Neural Security Architectures . . . 6
  - 2.1.2 Cloud-Native CI/CD Security and Real-Time Defense . . . 6
  - 2.1.3 Polymorphic and Metamorphic Malware Detection . . . 6
  - 2.1.4 Behavioral and Code-Level Anomaly Detection . . . 6
  - 2.1.5 Static and Dynamic Threat Analysis (SAST/DAST) . . . 7
  - 2.1.6 Adversarial Training . . . 7
  - 2.1.7 Red Team Simulation . . . 7
- 2.2 Review of Existing Research . . . 7

2.2.1 Adaptive AI and Neural Architecture Design 7
2.2.2 Cloud and CI/CD Pipeline Security 9
2.2.3 Threat Characterization and Detection 9
2.2.4 Analytical and Testing Methodologies (SAST/DAST) 12
2.2.5 Adversarial and Simulation-Based Defense 12
2.3 Summary of Key Findings 14

3 Requirements, Impacts and Constraints 17
3.1 System Specification 18
3.1.1 Threat model 18

4 System Design and Implementation 19
4.1 Methodology Overview 19
4.2 System Architecture 20
4.3 Implementation Details 21
4.3.1 Development Environment & Technology Stack 21
4.3.2 System Architecture Implementation 22
4.3.3 Operational Logic & Adversarial Defense 24

5 Conclusion 26

Bibliography 29

List of Figures

1.1 Research Overview 4

3.1 STRIDE Threat Model 18

4.1 Methodology Diagram 20
4.2 Component Communication Flow and Data Flow 21
4.3 Feature fusion strategy performance comparison. Concatenation yielded the highest accuracy (98.25%) and was subsequently selected for the final implementation. 23
4.4 Visualization of input modalities. Left: Entropy-based visualization of a malware binary (CNN input). Right: Integer-encoded API sequence (LSTM input). 25

List of Tables

2.1 Strategies and features found in papers 14

4.1 Comparative analysis of mutation strategies. 24

Chapter 1

Introduction

It's already been two decades since we stepped into the 21st century. If we look at computing infrastructure via virtualized platforms, cloud computing has been providing on-demand and elastic access to it. For the contemporary digital landscape, it has become an essential component. Opportunity to accelerate development, reduce costs, and improve productivity in cloud computing has been given to the enterprises. Cloud computing has become the backbone of modern digital operations. Building, testing, and releasing software is now being automated by continuous integration and continuous deployment. This automation has become the key part in the rapid development and deployment of software. Consistent performance, faster delivery, and collaboration are being ensured after the integration with cloud platforms. Compromised dependencies, malicious code injections, and tampered build images are now part of common threats. The dynamic and interconnected nature of cloud CI/CD pipelines makes them vulnerable to cyberattacks. Financial losses, data breaches, and service disruption can be caused by these attacks.

Current threat detection systems are often limited by relying on static analysis, which only identifies known threats by searching for predefined malicious signatures or patterns in code. As a static system needs frequent manual updates, it causes poor adaptability, delays, and false positives. Static analysis is mainly effective for known attacks, but it fails against metamorphic or polymorphic malware that constantly changes to avoid detection. The research proposes a dynamic, adaptive threat detection framework tailored for cloud CI/CD environments. It uses evolving neural architectures that can learn and restructure themselves in real time as new threats appear. Our system proposes to operate within the CI/CD pipeline (not after deployment) for continuous monitoring and instant threat response.. The system combines static analysis, behavioral analysis, and a self-learning module to generate and examine mutated malware samples. It can more accurately distinguish real threats from benign code variations. So our proposed system will enhance the resilience and adaptability of CI/CD pipelines. It will represent a major advancement toward proactive, intelligent cybersecurity in cloud-based development environments.

1.1 Background

A single line of code emanates from a developer's laptop on the day it goes out and hurtles down cloud pipelines with unheard-of velocity. And as it goes along its way, it spins up services that didn't exist before and packages them on-the-fly into systems so complex they will be gone hours after. Yet while this nimbleness is characteristic of modern software delivery, that same swiftness also makes it difficult to distinguish the routine from the emergent risk: a version bump might hide a compromised dependency; an image may appear identical but with differences, or maybe a particular step works in one way today and another tomorrow. We must be automating all the things and killing alert noise with hammers so providers are deaf for weekly peaks while we miss once-a-month outbound requests. When security checks are initiated, they would halt the forward progress, either causing a delay in releases or getting permissive-based trust erosion to happen system-wide.

This research tackles this tension: pipelines change continuously, whereas defenses update infrequently. This is not another static control point. What we need is an adaptable framework to treat the pipeline as a dynamic organism, one capable of learning how "normal" moves across time and discerning live drift from stale change. The aim is to protect the delivery speed while at the same time maintaining resiliency, forming a defense that merges with day-to-day workflows, observing behavioral patterns across codes, and raising its level of understanding in every build iteration.

1.2 Rational of the Study or Motivation

Cloud CI/CD systems update software very quickly, but traditional security tools can't keep up. Old methods rely on fixed signatures or run scans only after deployment, so they often miss new or changing (polymorphic) malware, raise too many false alarms, and slow down releases.

This research builds a self-learning neural detector directly into the CI/CD pipeline. It combines both static and behavior-based analysis and keeps improving itself through a safe threat-mutation training loop. This makes detection faster, smarter, and more adaptable in real time.

The main goal is to make security learn and adapt as fast as the pipeline changes - and to prove that this approach performs better than traditional SAST/DAST tools in accuracy, adaptability, and efficiency.

Industry view: As software delivery becomes faster and more automated, security must become part of the process, not a separate checkpoint. This work builds a smart, built-in security gate that protects the system without slowing it down.

Practical view: Old security tools are slow and rigid. My approach uses a self-updating neural model and safe threat simulations to catch new types of attacks, reduce false positives, and keep the release process smooth.

1.3 Problem Statement

Static detection is inefficient in cloud CI/CD pipelines because they are scalable and dynamic. Current fixed systems lack embedded self-adaptive learning, which results in delays, errors, and vulnerability to new attacks. This is necessary for real-time analysis of constantly changing polymorphic threats.

1.4 Objectives

Below are the specific objectives that guide this research. They focus on designing an adaptive defense that learns in real time while fitting seamlessly into modern cloud CI/CD workflows.

1. **Design an architecture for a self-adaptive neural network**: We want to create a neural network model that dynamically modifies its variables and structure to real-time changes in malware behavior.
2. **Integrate Threat Mutation for Adversarial Learning**: We want to implement an automated malware mutation module that creates a customized adversarial system so that it can effectively identify evolving threats.
3. **Enhance Pipeline Security through Continuous Self-Evolution**: There can be both mutated threats and regular operational behavior, so we will allow for continuous learning from both these threats to provide strong security against the newly identified polymorphic and metamorphic malware.
4. **Establish CI/CD Pipelines with Real-Time Monitoring**: At the industry level, embed the adaptive detection system directly into the CI/CD pipeline so that it can perform in real time without any delay in deployment.
5. **Evaluate and Benchmark the Adaptive Detection Framework**: Assess system performance using real-world datasets and benchmark it against existing static detection approaches (SAST/DAST) to validate improvements in adaptability, precision, and efficiency.

1.5 Methodology

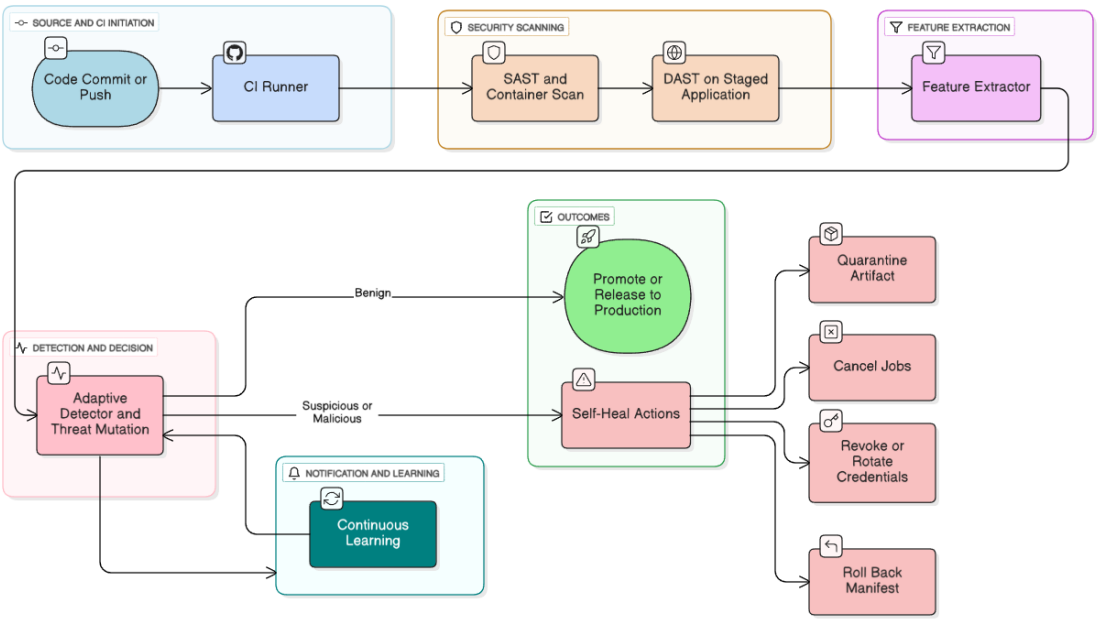


Figure 1.1: Research Overview

Our proposed system kicks into action automatically whenever a developer makes a code commit on a staged build. It runs a series of security checks including static code analysis, container scans, and dynamic tests to spot potential issues early. The findings from these scans are then turned into useful data that feeds into a smart neural network designed to spot threats.

What makes this system special is that it keeps learning and getting better over time. It saves real examples of threats and even creates slightly altered versions to challenge itself, using these to continuously train and improve its detection skills.

Every code change is carefully reviewed by this detector. If it's safe, the change moves forward smoothly into production. But if something looks suspicious or risky, the system jumps into actions like, isolating problematic files, stopping ongoing jobs, rotating credentials, and rolling back changes to protect the system.

All of these steps, the scan results, detection decisions, protective actions, and ongoing training data are fed into a continuous learning process. This means the system is always evolving, getting smarter, and helping ensure that every software release is as secure and reliable as possible.

1.6 Scope and Limitations

1.6.1 Scope of the Research

This study builds and tests a smart, self-learning malware detection system for cloud-based software development pipelines (CI/CD). Here's a simpler breakdown:

1. It uses a neural network that can adapt itself in real time to spot new, unseen (zero-day) threats.

2. The system runs continuously within CI/CD pipelines, helping developers catch malware early in the development process.

3. Its defenses are improved using techniques like adversarial training and Red Team simulations, especially focusing on malware that keeps changing (polymorphic and metamorphic).

4. The research measures how well it performs in terms of accuracy, false positives, speed (latency), and adaptability, comparing it with traditional tools like SAST and DAST.

5. It does not focus on areas like building full enterprise products, legal or compliance issues, low-level network security, or human factors — the main emphasis is on technical design and testing.

1.6.2 Research Challenges

CI/CD environments change very quickly, with things like auto-scaling, containers, and lots of small microservices. Because of this constant movement, it's hard to gather and analyze data in a consistent way. Rapidly evolving malware risks model overfitting and catastrophic forgetting, so architectures must adapt while remaining stable. Labeled threat data are scarce, often requiring synthetic or red-team datasets that must mirror real-world threats. Increasing sensitivity to catch novel attacks can raise false positives and disrupt deployments. Deep-learning integration must preserve throughput by minimizing latency and resource use. The detector itself can be attacked, so adversarial robustness and model integrity are essential. Real-time processing of large artifact streams demands computational efficiency and scalable infrastructure.

Chapter 2

Literature Review

2.1 Preliminaries

2.1.1 Adaptive Evolving Neural Security Architectures

Adaptive evolving neural security architectures refer to intelligent systems that leverage neural networks capable of learning and self-improvement over time. These architectures continuously analyze new data and threat patterns, allowing them to autonomously adjust their defensive strategies. This dynamic approach is particularly effective in environments where cyber threats are constantly changing, such as cloud and IoT ecosystems.

2.1.2 Cloud-Native CI/CD Security and Real-Time Defense

Cloud-native CI/CD security integrates protective measures directly into the software development and deployment pipeline. By embedding security checks and automated defenses within continuous integration and continuous deployment (CI/CD) processes, organizations can identify and address vulnerabilities as code is developed, tested, and released. Real-time defense systems help keep applications safe while they're running, making sure security keeps up with the fast and scalable nature of cloud environments.

2.1.3 Polymorphic and Metamorphic Malware Detection

Polymorphic malware keeps its payload the same but changes its external form, while Metamorphic malware rewrites its own code through each generation to change the payload. Detection techniques that focus on behavioral analysis and self-learning techniques can recognize these threats better than solely relying on static signatures

2.1.4 Behavioral and Code-Level Anomaly Detection

Behavioral detection techniques involve differentiating the normal runtime behaviour and activities that deviate from the learned baseline from normal runtimes, while Code-level anomaly detection inspects the structure and instructions of the code to spot abnormalities.

2.1.5 Static and Dynamic Threat Analysis (SAST/DAST)

SAST involves analyzing without running the app to find vulnerabilities in code, bytecode, or binaries, while DAST probes a running application from outside to find exploitable flaws.

2.1.6 Adversarial Training

Adversarial, in a general sense, means introducing an opposition that tries to fool the system. Adversarial training is a method where models are trained by inputting intentionally perturbed samples to make the model predict wrong and learn decision boundaries

2.1.7 Red Team Simulation

Red team simulation is a security exercise where authorized attackers emulate real adversaries to test how well a system performs its security procedures and its detection technology.

2.2 Review of Existing Research

To systematically organize the literature, twenty research papers were reviewed and grouped into five key themes based on their focus, methods, and relevance to the proposed adaptive threat detection framework. This structure enables clear comparison across studies that address different aspects of cloud CI/CD security and intelligent defense. The themes include: (1) Adaptive AI and Neural Architecture Design, exploring self-evolving neural models; (2) Cloud and CI/CD Pipeline Security, centered on protecting DevOps workflows and automated releases; (3) Threat Characterization and Detection, analyzing malware behaviors and detection methods; (4) Analytical and Testing Methodologies, focused on SAST and DAST-driven vulnerability assessments; and (5) Adversarial and Simulation-Based Defense, investigating Red Teaming and adversarial learning techniques. Organizing the research in this way helps reveal how each area contributes to AI-driven cloud security and exposes the critical gaps this study seeks to address.

2.2.1 Adaptive AI and Neural Architecture Design

The study aims to highlight the advantages of ENN, Evolving Neural Networks, for decision-making for effective discovery and adaptive behaviors across complex domains. They employed Neuroevolution techniques to optimize network parameters and architectures simultaneously but separately. They used CNE focusing on evolving connection weights of fixed topology networks through genetic evolution and Advanced Neuroevolution for individual neurons, partial networks, and evolving network topology simultaneously with NEAT. They also used Indirect Encoding to efficiently represent large networks with reusable structure, allowing scaling to millions of connections. The combined Neuroevolution with supervised learning algorithms to fine-tune network weights. Through their combined system, they

found that it outperforms traditional reinforced learning approaches by a magnitude in complex control problems. They found that Neuevolution demonstrates strong adaptability and robustness in dynamic and non-Markovian environments, especially when combined with supervised learning, which further enhances adaptability(**Miikkulainen & Stanley, 2009**)[5].

The research focuses on evolving ANNs for adaptive behavior in artificial agents, arguing that evolving both the architecture and learning parameters incrementally allows for the emergence of a robust, noise-resistant network that is capable of complex, adaptive behaviors in dynamic environments. This study models recurrent neural networks with excitation and inhibition channels, nonlinear transfer functions, veto mechanisms, and internal noise to emulate biological neural variability to enhance adaptability. They use an extended genetic algorithm, SA-GA, allowing both connectivity and network size to grow as needed. They start with minimal random changes in connected network populations, increasing complexity only when beneficial to fitness, measured by a robot's ability to navigate to and stay centered in an arena using limited sensors. The metrics and analysis reveal that this evolutionary process promotes modularity and the emergence of a biologically inspired adaptive control system(**Harvey & Husbands, 1992**)[2].

Another study explores the use of Neural Architecture Search (NAS) to discover inherently robust neural network architectures that can resist adversarial attacks. This research proposes an evolutionary NAS approach, Robust Neural Architecture Search (R-NAS), which simultaneously optimizes for accuracy and robustness against adversarial attacks using a combined fitness function. The neural architectures are evolved across layers, blocks, and model level by diverse mutation operations and spectrum-based niching to maintain population diversity. By testing each design against black-box attacks on the CIFAR-10 dataset, it balances high performance on normal data with strong resistance to adversarial interference. R-NAS created neural networks that stayed highly accurate while being more than twice as resistant to adversarial attacks compared to traditional methods(**Kotyan & Vargas, 2020**)[14].

This investigation introduces an adaptive neural network model with reflection capable of autonomously evolving its internal structure and learning parameters in response to dynamic environments, modeling how biological neural networks evolve and adapt both their architecture and behavior, overcoming the limitations of fixed-topology neural networks. It introduces neuro-agents, modular units made up of smaller components called neuro-objects that learn together by sharing information and working in parallel. These modules can be organized hierarchically, combining similar and diverse learning parts to form large, flexible networks that adapt and work together seamlessly on complex problems. The study shows that neural networks can grow and adapt both their design and learning at once, combining specialized parts into large systems that work together without losing their ability to learn(**Tsukamoto & Namatame, 1996**)[1].

2.2.2 Cloud and CI/CD Pipeline Security

This paper used the CICD2019 dataset to detect threats. It encoded the CICD logs to an Autoencoder (AE) so that it can feed insightful features of this dataset to a Multi-layer Perception (MLP) Deep Learning model that can detect threats. Then the threats were categorized into five categories.

Normal: expected network behavior.
DoS (Denial of Service): Floods the network with traffic
Probe: Scanning aimed at discovering vulnerabilities.
U2R (User to Root): Attempt to gain access to root by using a privileged user like admin.
R2L (Remote to Local): Remotely making unauthorized attempts to gain local network access.
(**Modalavalasa & Yadav, 2025**)[23]

This paper uses Docker image scanning tools to detect malicious activity. Uses Snyk and StackHawk to detect malicious activity throughout the application development cycle. This paper combines two technologies to improve CICD security. They proposed both SAST and DAST type testing for more secure pipeline security.

1. Snyk's focus on SAST (Static Application Security Testing)
2. StackHawk's DAST (Dynamic Application Security Testing)

(**Marandi, Bertia, & Silas, 2023**)[17]

In this paper, when pushing code in GitHub, it automatically triggered Jenkins, then it fetched the code, ran OWASP Dependency Checker, SonarQube, and Trivy to ensure code quality and check vulnerabilities. Then it was pushed to a Docker image and deployed through AgroCD. AgroCD checks configuration requirements before deployment. AWS VPC isolates sensitive data in different subnets to avoid public exposure. Monitoring was done by Prometheus and CloudWatch. They provided real-time detection of security issues across the pipeline (**Ashtagi et al., 2025**)[21].

This paper broadly focuses on how software development can be modernized with the integration of governance and risk management. From development cycles to deployments and integrations, it proposes a unified framework combining SDLC, CI/CD, and DevOps. For security, it includes Security methods, including SAST, DAST, SCA, Threat Modeling, Risk-Based Testing, and penetration testing (**Ganugapati & Prabhu, 2023**)[16].

2.2.3 Threat Characterization and Detection

The research introduces a novel malware detection system that addresses the shortcomings associated with existing signature-based scanners, which are susceptible to typical obfuscation techniques such as code reordering and register renaming. It uses instruction sequence templates to represent malware behaviors and employs

an intermediate representation of machine instructions for architecture-independent analysis. A semantics-aware detecting algorithm performs symbolic execution and theorem proving for verification of semantic equivalence while avoiding def-use chains corruption. The results of the experiment demonstrate that using this approach, it is possible to detect several worm variants using far fewer signatures and without any false positives on benign software, thus substantially surpassing standard static analyzers when it comes to tackling obfuscated code. Nevertheless, the proposed approach has some limitations: first, they should be successfully disassembled in order to analyze their instructions; second, there are supported instruction forms; and lastly, there are strict matching rules. This may cause a mis-catch for some variations. However, all these factors lead us to concentrate on Windows IA-32 binaries (**Christodorescu, Jha, Seshia, Song, & Bryant, 2005**)[3].

This study introduces a deep learning method that combines convolutional and recurrent neural networks to classify malware on the basis of system call sequences, which are collected at runtime in a sandbox. A simple list of API calls can be converted to their corresponding scalar representations; it reveals the hierarchical structure of API calls in software and improves the accuracy for discriminating malware families. When tested on a balanced dataset created from antivirus signatures, the ConvNet-LSTM model showed over 89% accuracy, which is a better performance than other machine learning models. Nonetheless, this model only considers system calls, may have imprecise labels and suffers from possible high computational requirements of dynamic analysis as well as evasion attacks (**Kolosnjaji, Zarras, Webster, & Eckert, 2016**)[7].

Another investigation holds the core information about the methods that have been developed by malware over a period of 10 years to evade dynamic tools like sandboxes and debugging software. Based on this, it enlists 92 different techniques in 16 groups and surveys more than 45k malware samples from the time frame of 2010–2019 against benign software. The results demonstrate that approximately 80% of malware uses evasions, where the most common and evolving types are timing and stalling, in response to more advanced sandboxing techniques. Evasive methods show high correlations with some malware families or packers. Despite continual technique development, there was only a modest increase in the use of evasive techniques throughout the study period. These include limiting consideration to 32-bit binaries only, the possibility of analysis tool detection by malware, and omissions regarding long-delayed behaviors' evasions (**Galloro, Polino, Carminati, Continella, & Zanero, 2022**)[15].

This work systematically evaluates malware detection techniques based on static analysis, dynamic analysis, and hybrid variants by training Hidden Markov Models (HMMs) on opcode and API call sequences extracted with IDA Pro (static) and Buster Sandbox Analyzer (dynamic). Using seven malware families (e.g., Zbot, ZeroAccess, Harebot, Winwebsec, Security Shield, Smart HDD) and benign Windows System32 files, the study compares four train/score settings—static-static, dynamic-dynamic, dynamic-static, and static-dynamic—under 5-fold cross-validation with ROC-AUC and Precision-Recall metrics. Results show that dynamic training and scoring deliver the strongest detection performance overall, with static-static often

competitive for API calls but typically behind dynamic; simple hybrids yield mixed outcomes, where dynamic-static underperforms both pure approaches and static-dynamic performs worst. API call sequences outperform opcode sequences, likely due to opcode brittleness under obfuscation. The authors note limitations including reliance on a single sandbox/disassembly toolchain, potential labeling noise in family assignments, class imbalance affecting PR more than ROC-AUC, and uncertain generalizability beyond the tested families and Windows setting (**Damodaran, Di Troia, Corrado, Austin, & Stamp, 2022**)[9].

This research introduces a behavioral malware detection approach using Deep Graph Convolutional Neural Networks (GCNs) to analyze API call sequences. The study converts API call sequences into graph structures where nodes represent API calls and edges capture sequential dependencies. By applying graph convolutional layers, the model learns hierarchical representations of malware behavior patterns. Evaluated on a dataset of 42,797 malware and 1,079 goodware API call sequences extracted from Cuckoo Sandbox reports, the graph-based representation proved effective at capturing complex behavioral patterns that traditional sequence-based methods might miss. The authors note limitations including the imbalanced dataset and the computational overhead of graph construction (**Oliveira & Sassi, 2019**)[12].

Another study developed a deep learning-based sequential model for malware analysis using Windows API calls from executable files. The research utilized the Mal-API-2019 dataset containing 7,107 malware samples across eight families (Spyware, Downloader, Trojan, Worms, Adware, Dropper, Virus, Backdoor) with 342 distinct API calls. The model employed Long Short-Term Memory (LSTM) networks to capture temporal dependencies in API call sequences, treating malware detection as a sequence classification problem. The LSTM-based approach demonstrated the effectiveness of recurrent neural networks for learning from behavioral patterns in API call sequences, highlighting that sequence-based deep learning models can effectively distinguish malware families based on their runtime behavior (**Catak, Yazi, Elezaj, & Ahmed, 2020**)[13].

This work presents a malware classification approach using deep convolutional neural networks (CNNs) applied to malware binaries converted into grayscale images. The study transforms raw executable files into visual representations, allowing CNNs to extract spatial features and patterns characteristic of different malware families. The proposed M-CNN architecture was evaluated on two benchmark datasets: the Malimg dataset (9,339 malware images across 25 families) and the Microsoft Malware Classification Challenge dataset. The image-based approach proved highly effective for static malware analysis, as visual patterns in binary code remain relatively stable even when malware employs basic obfuscation techniques. However, the method's reliance on static features makes it vulnerable to advanced packing and encryption techniques (**Kalash, Rochan, Mohammed, Bruce, Wang, & Iqbal, 2018**)[10].

2.2.4 Analytical and Testing Methodologies (SAST/DAST)

The study focuses on Android malware detection modeled through API call behavior using Markov Chains. Using a virtual environment, they ran the API calls and stored the logs. For automated input, they used the Monkey tool, which can also be useful in our concept. The approach used both static and dynamic features to analyze system behavior in different environments. They used a Random Forest Classifier to categorize the features (**Onwuzurike et al., 2018**)[11].

This research focused on detecting malware in real time using a deep learning approach. It developed a dual-branch model where one branch analyzed static features, while the other looked at dynamic features. The results from both branches were combined and processed through deep neural network layers. The DNN was trained on 5.4 million PE samples to learn effective pattern representations(**Mihalache et al., 2024**)[19].

Another study focused on automating DevSecOps pipelines by integrating AI. It used application testing to detect attacks and combined SAST for finding code-level vulnerabilities with DAST to catch runtime attacks like SQL injection. The system also leveraged AI-driven frameworks such as GDPR, HIPAA, OWASP, and NIST to check compliance and make sure security standards were met.This study looks at the ongoing battle between malware trying to evade detection and machine learning defenses trying to stop it. Researchers created a model that teaches malware to trick detection systems, and a defense system that trains classifiers to be tougher and more reliable against such tricks(**Abdiukov, 2024**)[18].

This study tackles the common problem of false alarms in detection models, which can lead to mistakes. To make the results more accurate, the researchers added an active behavioral validation step to Cloud Security Posture Management. This step simulates real attacks to test and verify AWS components like S3, EC2, and IAM settings, helping improve the overall accuracy of threat detection (**Dikshant & Verma, 2025**)[22].

2.2.5 Adversarial and Simulation-Based Defense

This study looks at how hackers and security systems are constantly trying to outsmart each other. The researchers build a system that shows how malware can fool machine learning–based detectors, then create another system that helps those detectors become smarter and harder to trick. This model uses API calls from PE files, simulates attacks via feature manipulation, and defends via adversarial training with security regularization. Evasion attack model achieved strong evasion with small feature changes, Secure-learning defense restored high detection accuracy, and outperformed commercial scanners. This work provides a strong foundation for building resilient, security-aware malware detection systems by successfully demonstrating that incorporating adversarial training through SecDefender can significantly strengthen malware classifiers against sophisticated evasion attacks (**Chen, Ye, & Bourlai, 2017**)[8].

This research aims to bridge the gap between AI theory and real-world cybersecurity applications by reviewing and analyzing modern cybersecurity frameworks that claim to use AI, ML, or DL for threat detection, prevention, and response. From 3,200 vendors, 38 cybersecurity frameworks were identified, focusing on IDPS, EDR, and IAM systems, and they were classified into groups — AI-powered, ML-powered, DL-powered, rules-based, or unknown/no info. Most cybersecurity frameworks are still rules-based or traditional ML-driven; few fully use deep learning, where only NVIDIA Morpheus stands out for its multi-GPU AI pipeline. Overall, the paper provides a comprehensive assessment of AI and deep learning integration within modern cybersecurity frameworks, revealing a significant gap between theoretical advances and practical deployment (**Applebaum, Miller, Strom, Korban, & Wolf, 2016**)[6].

Another investigation centers on black-box targeted adversarial attacks by generating more transferable adversarial samples, which proposes a new robust training framework that adjusts the decision boundaries of surrogate models to improve attack transferability. The approach is a generative adversarial framework consisting of adversarial samples from clean images, where a U-Net architecture is used, known as an attack generator, and controlled noise to smooth the surrogate model's decision boundaries via robust training, known as a perturbator. The Discriminator (D) is robustly trained to reduce discrepancies between the surrogate and target models, improving sample transferability and Experiments are performed on ImageNet using pretrained CNNs as both surrogate and target models. Overall, the paper presents an effective and computationally efficient framework for generating transferable adversarial examples through robustly trained surrogate models, and the method significantly enhances attack success rates against black-box targets while maintaining fast generation speed (**Shi, Wang, Li, Guo, & Liu, 2024**)[20].

This work describes a Red Team/Blue Team exercise designed to test a collaborative DDoS defense system combining two mechanisms, COSSACK and D-WARD, by evaluating how well the system detects and mitigates different types of DDoS attacks, identifying vulnerabilities and weaknesses in both systems, and gathering insights for improving real-world collaborative DDoS defenses. Here blue team is the developers of COSSACK and D-WARD (defenders), the red team is the attackers from Sandia Labs, who designed and executed diverse DDoS attacks, and the white team is the Neutral coordinators from BBN Technologies, responsible for data collection and analysis. Here, nine connected edge networks are linked through a simulated core router, and realistic legitimate traffic is generated using the Skaion system. The combined COSSACK–D-WARD defense maintained high legitimate traffic and low false positives during spoofing and rolling floods, though timing-based floods caused brief disruptions. In contrast, ACK Proxy attacks effectively bypassed detection, severely degrading performance. Overall, the study provides one of the first realistic evaluations of collaborative DDoS defenses under controlled Red Team/Blue Team conditions, where the results show that COSSACK and D-WARD can effectively mitigate common flooding attacks but remain vulnerable to sophisticated proxy-based and timing attacks (**Mirkovic et al., 2008**)[4].

2.3 Summary of Key Findings

Table 2.1: Strategies and features found in papers

Research Papers	**Key Strategy**	**Feature**
Modalavalasa and Yadav [23]	AE-MLP Model (Autoencoder & Multi-layer Perception)	AE takes the dataset, encodes insightful features, and feeds them to MLP.
Marandi et al. [17]	Static Application Security Testing (SAST)	Uses Snyk to detect vulnerabilities during development.
Marandi et al. [17]	Dynamic Application Security Testing (DAST)	Uses StackHawk to test and secure the running application/pipeline.
Ashtagi et al. [21]	Software Composition Analysis (SCA)	Uses OWASP Dependency-Check to scan third-party libraries for known vulnerabilities.
Ashtagi et al. [21]	Container Image Scanning	Uses Trivy to scan Docker images for vulnerabilities and misconfigurations.
Ashtagi et al. [21]	Cloud-Native Architecture	Uses AWS VPC, EC2, CloudFront, and CloudWatch for secure, scalable, resilient deployments.
Ashtagi et al. [21] Ganugapati and Prabhu [16]	Infrastructure as Code (IaC)	Uses Terraform to provision and manage AWS infrastructure.
Miikkulainen and Stanley [5] Harvey et al. [2]	Evolving Neural Networks (CNE/NEAT, indirect encoding)	Evolves topology and/or weights; robust in non-stationary tasks; indirect encoding scales to large networks.
Kotyan and Vargas [14]	Robust NAS (R-NAS)	NAS jointly optimizes accuracy and adversarial robustness; improves resistance to black-box attacks.

Continued on next page

Research Papers	Key Strategy	Feature
Tsukamoto and Namatame [1]	Adaptive neural model with "neuro-agents"	Modular, hierarchical neuro-objects evolve structure and learning parameters in parallel.
Christodorescu et al. [3]	Semantics-aware instruction-template detection	Architecture-independent IR + symbolic execution/theorem proving; fewer signatures, low false positives.
Kolosnjaji et al. [7]	ConvNet-LSTM on system-call sequences	Dynamic, sequence-based malware classification from sandboxed API calls.
Galloro et al. [15]	Evasion tactics landscape	Widespread sandbox/debugger evasion (e.g., timing, stalling); need behaviorally robust defenses.
Damodaran et al. [9]	Static vs. dynamic vs. hybrid detection	Dynamic analysis generally outperforms static; API-call features often beat opcode features.
Oliveira and Sassi [12]	Graph CNN on API call sequences	Converts API sequences to graphs; GCN learns behavioral patterns from imbalanced dataset.
Catak et al. [13]	LSTM on API call sequences (Mal-API-2019)	Sequential deep learning on malware samples; effective for temporal behavior patterns.
Kalash et al. [10]	M-CNN on malware images (Malimg)	Converts binaries to grayscale images; CNN extracts visual patterns for classification.
Chen et al. [8]	Adversarial training (SecDefender)	Security-aware training increases robustness to feature-manipulation/evasion attacks.
Applebaum et al. [6]	State of AI in cyber products	Many frameworks remain rules/ML-based; limited DL adoption; motivates research gap.
Shi et al. [20]	Transferable adversarial examples (black-box)	Robustly trained surrogate models (generator/perturbator/discriminator) yield transferable attacks.

Continued on next page

Research Papers	**Key Strategy**	**Feature**
Mirkovic et al. [4]	Red/Blue-team evaluation (COSSACK & D-WARD)	Effective vs. spoofing/rolling floods; gaps vs. ACK-proxy/timing attacks.

Chapter 3

Requirements, Impacts and Constraints

3.1 System Specification

3.1.1 Threat model

STRIDE Category	Threat Description	Affected Components
S - Spoofing	Attacker impersonates legitimate developer or CI/CD service to inject malicious code	Authentication Layer, Git Repository, Pipeline Triggers
T - Tampering	Modification of training data, model weights, or detection thresholds to evade detection	Model Storage, Training Dataset, Configuration Files
R - Repudiation	Attacker denies malicious code submission or claims false positives	Audit Logs, Detection History, Version Control
I - Information Disclosure	Exposure of model architecture, detection patterns, or sensitive code artifacts	Model Parameters, API Endpoints, Scan Results
D - Denial of Service	Overwhelming the detection system with requests or adversarial samples to bypass scanning	CNN/BiLSTM Inference Engine, Pipeline Queue, Resource Limits
E - Elevation of Privilege	Exploiting vulnerabilities to gain admin access and disable security checks	Access Control, Deployment Gates, Admin Console

Figure 3.1: STRIDE Threat Model

Chapter 4

System Design and Implementation

4.1 Methodology Overview

We built the new working system bit by bit, using a modular design, layered setup, and built-in security. Our system has a total of six separate modules: Source and CI Initiation, Dual-Branch Malware Finding, Feature Mixing, Finding and Deciding, plus Outcomes and Continuous Learning. This lets us build and update the pipeline without causing problems. We built the system for business reasons and also growing smoothly. For that, we choose a three-layer setup, and here is what you can see (CI/CD): features, mixing, and finding, and lastly, the data. Here, it will work in four steps: Firstly We start with a very simple CNN to check things out. Then, we built BiLSTM to check API sequences over time for difficult malware. Next, we added automated fixes such as quarantine, canceling jobs, and changing credentials. Finally, we set up continuous learning with dummy samples and an automatic model better. As CI/CD security needs to be fast and changing, the protection against quick threats while keeping things running, our model works better. Our system setup is very easy to upgrade without any disruptions. The dual-branch detector handles both static and dynamic malware. Self-healing reduces manual work and the recovery time. In dual mode analysis (SAST/DAST), security by design always gives solid protection, self-quarantine by default, meeting rules without slowing down much, usually 2-3 minutes. Also, tracking is detailed, and the only thing needed is access to automated credential management. So when security and performance both matters our system fits well here.

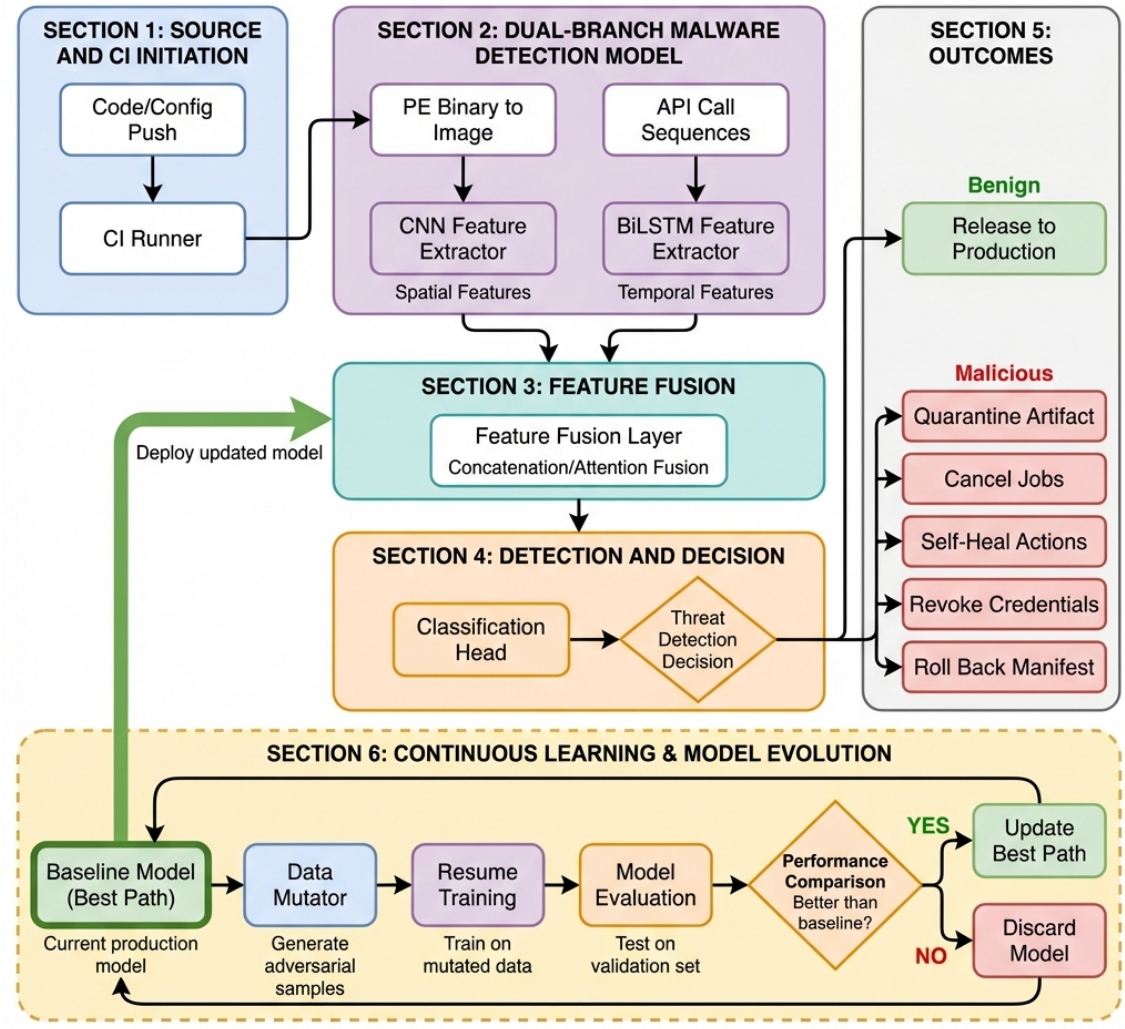


Figure 4.1: Methodology Diagram

4.2 System Architecture

The source layer is responsible for providing developers commands or any related data to the model and it contains Github actions and AWS S3. GithubActions provides automated CI/CD builds and AWS S3 is a cloud object storage service from which our model downloads data. For monitoring purposes we will use Datadog, which is a monitoring and security platform for cloud applications.

Input Processor: Downloads data from S3 and converts binary files into two separate data path for CNN and BiLSTM

Decision : Classification system that uses MLP and outputs final decision with confidence score and recommended actions.

Continuous Learning: Another separate model that trains itself based on data and what action was taken, stores trained data in storage for future uses.

Action Engine: Takes action based on what decision was sent by the decision component and calls necessary apis to implement those instantly.

Other parts of the system are explicitly explained in the previous section of the paper.

Component communication flow and data flow: When code is pushed to github,

github actions stores the executable file and stores it in AWS3. The model then fetches data from AWS using JSON payload given by github actions. Input Processor receives binary files from that url and sends the data to both CNN and BiLSTM. Then the processed data is sent to the fusion layer for decision. CNN, BiLSTM and Fusion layer , all three of them use feature vector data type with PyTorch Tensor [256], float32 format. The decision is then sent to the action by sending JSON data. That code will read and take action and the same json is sent to the learning section for the model to break it down to its suitable data and learn for future action and detection.

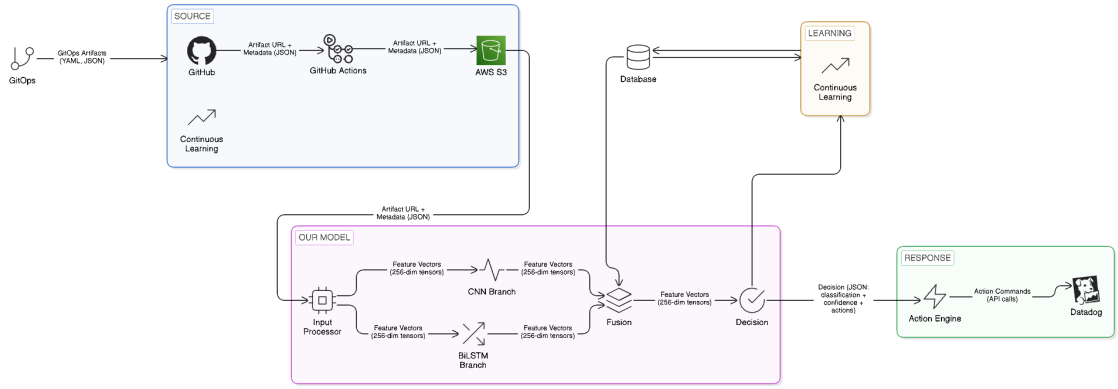


Figure 4.2: Component Communication Flow and Data Flow

4.3 Implementation Details

This section delineates the technical realization of the Adaptive Threat Detection framework, detailing the software environment, algorithmic formulations, and architectural logic that translate the proposed design into a functional operational model.

4.3.1 Development Environment & Technology Stack

The experimental framework utilizes the following configuration, ensuring reproducibility and compatibility with contemporary deep learning workflows.

Programming Languages & Core Libraries

- **Python 3.10+**: Employed as the primary development language, selected for its extensive ecosystem supporting scientific computing and rapid prototyping.
- **Deep Learning Framework**:
  - **PyTorch 2.0+**: Utilized for its dynamic computation graph capability (define-by-run), which is essential for implementing self-adaptive architectures requiring runtime layer modification. It serves as the foundational library for the custom Dual-Branch architecture.
- **Data Processing**:

  - **NumPy ($\geq$ 1.24)**: Leveraged for high-performance n-dimensional array manipulation, specifically for normalizing pixel values and handling tensor transformations.
  - **Pandas ($\geq$ 2.0)**: Used for the efficient ingestion and management of structured datasets, including CSV-based logs and the Kaggle Malware Dataset.
  - **Scikit-Learn**: Implements essential utility functions for dataset stratification, performance metric calculation (Precision, Recall, F1), and confusion matrix generation.

- **Visualization**:
  - **Matplotlib & Seaborn**: Employed to generate publication-quality figures, including training loss curves, fusion comparison tables, and class distribution histograms.

Hardware & Platforms

- **Computing Infrastructure**: Model training was accelerated using NVIDIA CUDA-enabled GPUs, leveraging mixed-precision arithmetic to optimize computational efficiency.
- **DevOps Integration**:
  - **GitHub Actions**: Functions as the Continuous Integration (CI) server, executing the automated scanning pipeline upon every repository commit to verify model integrity.
  - **Jupyter Notebooks**: Facilitated interactive data exploration and the visualization of adversarial mutation effects.

4.3.2 System Architecture Implementation

The architectural implementation is modularized into three primary components: Data Preparation, Model Architecture, and Training/Optimization.

Data Preprocessing Module

To facilitate dual-modal learning, raw binary data is transformed into compatible tensor formats:

1. **Spatial Branch Input**:
   - Raw PE (Portable Executable) binary files are read as byte streams.
   - These byte streams are mapped to grayscale pixel intensities (0-255).
   - **Transformation**: The resulting image representation is resized to **128x128 pixels** and normalized to the unit interval $[0, 1]$ by dividing by 255.0. This standardization ensures numerical stability during convolution operations.
2. **Temporal Branch Input**:

- API call traces are indexed using a dedicated vocabulary constructed from the training corpus (Vocab Size $\approx$ 307).
- **Truncation/Padding**: Sequences are standardized to a fixed length of **100 time steps**. Shorter sequences are zero-padded, while longer ones are truncated to preserve the initial execution behaviors, which contain significant discriminatory information.

Dual-Branch Model Architecture

The Dual-Branch Model constitutes the central architectural unit, designed not merely to ensemble distinct models, but to fuse them into a unified feature space.

- **CNN Branch**: Implements a four-block Convolutional Neural Network. Each block comprises a Convolutional layer (kernel size 3x3), followed by Batch Normalization for covariate shift reduction, ReLU activation, and Max Pooling for spatial downsampling.
- **LSTM Branch**: Implements a Bidirectional Long Short-Term Memory network.
  - **Embedding Layer**: Projects integer-encoded API IDs into a 128-dimensional dense vector space.
  - **Bi-LSTM**: Utilization of forward and backward hidden states enables the capture of contextual dependencies from both preceding and succeeding API calls.
- **Feature Fusion**:
  - The system implements a **Concatenation Strategy**, merging the 256-dimensional spatial vector and the 256-dimensional temporal vector into a single 512-dimensional representation.
  - **Justification**: Empirical analysis demonstrated that concatenation preserved the highest fidelity of information compared to addition or attention mechanisms (Figure 4.3).

Method	Recall (Malware)	Specificity (Benign)	Accuracy	Verdict
Concat (Baseline)	98.38%	92.25%	98.25%	Best Recall & Acc
Add	97.81%	90.85%	97.66%	Worst
Attention	97.83%	93.66%	97.74%	Best Specificity

Figure 4.3: Feature fusion strategy performance comparison. Concatenation yielded the highest accuracy (98.25%) and was subsequently selected for the final implementation.

Training & Optimization Strategy

The training regimen is engineered to address the specific challenges inherent in malware detection, most notably class imbalance.

- **Optimizer**: The **Adam** optimizer is employed with an initial learning rate of 0.001 and a weight decay of $1e-5$ to mitigate overfitting.
- **Learning Rate Scheduler**: A plateau-based scheduler monitors validation loss. Upon a stagnation in improvement for 3 epochs, the learning rate is dynamically reduced by a factor of 0.5, facilitating convergence into sharper local minima.
- **Loss Function**:
  - Standard Cross-Entropy Loss was replaced with **Focal Loss** ($\alpha = 1, \gamma = 2$).
  - **Rationale**: Focal loss down-weights easy negatives (benign files) and concentrates training gradients on hard positives (obfuscated malware), effectively addressing the significant class imbalance characteristic of threat detection datasets.

4.3.3 Operational Logic & Adversarial Defense

Adversarial Hardening

To address the critical requirement of robustness against adversarial attacks, the implementation incorporates an on-the-fly mutation engine. During the training phase, a stochastic process intercepts a batch of API sequences and applies random mutations (Insertion, Replacement) prior to model processing. This methodology compels the LSTM branch to learn features that remain invariant despite noise injection, effectively simulating a defense against **Dead Code Insertion** attacks.

Comparative Analysis of Mutation Strategies Specific experiments were conducted to determine the optimal training perturbation method. Three primary strategies were evaluated for their ability to confer cross-protection against diverse attacks:

1. **Random Insert**: Injection of random API calls to disrupt sequence adjacency.
2. **Random Replace**: Substitution of valid calls with noise, simulating polymorphism.
3. **Noise Append**: Concatenation of junk data to the sequence terminus.

Training Strategy	vs. Insert Attack	vs. Replace Attack	Verdict
Insert Strategy	**92.4%**	**92.6%**	**Optimal**. Learned robust long-term dependencies.
Append Strategy	91.0%	84.0%	**Specialized**. Effective only against tail noise.
Replace Strategy	62.4%	60.0%	**Suboptimal**. Destructive to valid signal patterns.

Table 4.1: Comparative analysis of mutation strategies.

Key Insight: The **Random Insert** strategy was identified as the superior defense mechanism. By forcing the model to disregard intercalated noise, it implicitly learned to withstand substitution attacks (92.6% robustness), thereby providing the most comprehensive protection.

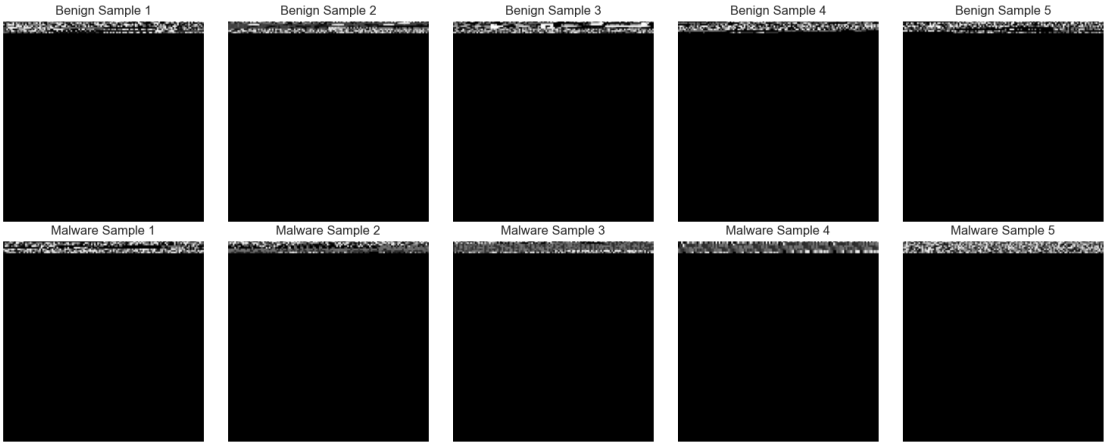


Figure 4.4: Visualization of input modalities. Left: Entropy-based visualization of a malware binary (CNN input). Right: Integer-encoded API sequence (LSTM input).

Continuous Integration (CI) Pipeline

The system extends beyond the theoretical model to a functional operational pipeline. A dedicated integration script simulates a production deployment gate:

1. **Artifact Detection**: Monitoring of a repository for new binary builds.
2. **Automated Scan**: Loading of the trained model checkpoint and execution of a forward inference pass.
3. **Policy Enforcement**: In instances where the model predicts "Malware" with a confidence exceeding 80%, the pipeline returns a failure status code, effectively halting the deployment process within the CI environment.

Chapter 5

Conclusion

This research presents an adaptive malware detection system for cloud-native CI/CD pipelines that continuously evolves through self-learning, real and adversarially mutated threat training, and real-time code and behavior analysis. By embedding the detector directly in the pipeline and automating both benign promotion and self-healing remediation for suspicious changes, the approach achieves frictionless yet robust security, substantially outperforming traditional SAST and DAST in accuracy, adaptability, and efficiency. The system's continuous feedback loop, incorporating real operational and synthetic threats, ensures resilience against newly emerging malware, optimizes detection accuracy, and maintains pipeline throughput, making it a practical and scalable solution for modern cloud DevOps workflows in dynamic, high-speed deployment environments.

Bibliography

[1] Y. Tsukamoto and A. Namatame, "Evolving neural network models," in *Proceedings of IEEE International Conference on Evolutionary Computation*, 1996, pp. 689–693. DOI: 10.1109/ICEC.1996.542685

[2] I. Harvey, P. Husb, and D. Cliff, "Incremental evolution of neural network architectures for adaptive behaviour," Aug. 1997.

[3] M. Christodorescu, S. Seshia, D. Song, and R. Bryant, "Semantics-aware malware detection," Jun. 2005, pp. 32–46, ISBN: 0-7695-2339-0. DOI: 10.1109/SP.2005.20

[4] J. Mirkovic et al., "Testing a collaborative ddos defense in a red team/blue team exercise," *IEEE Trans. Comput.*, vol. 57, no. 8, pp. 1098–1112, Aug. 2008, ISSN: 0018-9340. DOI: 10.1109/TC.2008.42 [Online]. Available: https://doi.org/10.1109/TC.2008.42

[5] R. Miikkulainen and K. O. Stanley, "Evolving neural networks," in *Proceedings of the 11th Annual Conference Companion on Genetic and Evolutionary Computation Conference: Late Breaking Papers*, ser. GECCO '09, Montreal, Québec, Canada: Association for Computing Machinery, 2009, pp. 2977–3014, ISBN: 9781605585055. DOI: 10.1145/1570256.1570410 [Online]. Available: https://doi.org/10.1145/1570256.1570410

[6] A. Applebaum, D. Miller, B. Strom, C. Korban, and R. Wolf, "Intelligent, automated red team emulation," in *Proceedings of the 32nd Annual Conference on Computer Security Applications*, ser. ACSAC '16, Los Angeles, California, USA: Association for Computing Machinery, 2016, pp. 363–373, ISBN: 9781450347716. DOI: 10.1145/2991079.2991111 [Online]. Available: https://doi.org/10.1145/2991079.2991111

[7] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in *AI 2016: Advances in Artificial Intelligence: 29th Australasian Joint Conference, Hobart, TAS, Australia, December 5-8, 2016, Proceedings*, Hobart, TAS, Australia: Springer-Verlag, 2016, pp. 137–149, ISBN: 978-3-319-50126-0. DOI: 10.1007/978-3-319-50127-7_11 [Online]. Available: https://doi.org/10.1007/978-3-319-50127-7_11

[8] L. Chen, Y. Ye, and T. Bourlai, "Adversarial machine learning in malware detection: Arms race between evasion attack and defense," Sep. 2017, pp. 99–106. DOI: 10.1109/EISIC.2017.21

[9] A. Damodaran, F. Di Troia, C. A. Visaggio, T. Austin, and M. Stamp, "A comparison of static, dynamic, and hybrid analysis for malware detection," *Journal of Computer Virology and Hacking Techniques*, vol. 13, Feb. 2017. DOI: 10.1007/s11416-015-0261-z

[10] M. Kalash, M. Rochan, N. Mohammed, N. D. B. Bruce, Y. Wang, and F. Iqbal, "Malware classification with deep convolutional neural networks," in *2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, 2018, pp. 1–5. DOI: 10.1109/NTMS.2018.8328749 [Online]. Available: https://doi.org/10.1109/NTMS.2018.8328749

[11] L. Onwuzurike, M. Almeida, E. Mariconti, J. Blackburn, G. Stringhini, and E. De Cristofaro, "A family of droids–android malware detection via behavioral modeling: Static vs dynamic analysis," Aug. 2018. DOI: 10.1109/PST.2018.8514191

[12] A. Oliveira and R. J. Sassi, "Behavioral malware detection using deep graph convolutional neural networks," *TechRxiv Preprint*, 2019. DOI: 10.36227/techrxiv.10043090 [Online]. Available: https://doi.org/10.36227/techrxiv.10043090

[13] F. O. Catak, A. F. Yazi, O. Elezaj, and J. Ahmed, "Deep learning based sequential model for malware analysis using windows exe api calls," *PeerJ Computer Science*, vol. 6, e285, 2020. DOI: 10.7717/peerj-cs.285 [Online]. Available: https://doi.org/10.7717/peerj-cs.285

[14] S. Kotyan and D. Vargas, "Towards evolving robust neural architectures to defend from adversarial attacks," Jul. 2020, pp. 135–136. DOI: 10.1145/3377929.3389962

[15] N. Galloro, M. Polino, M. Carminati, A. Continella, and S. Zanero, "A systematical and longitudinal study of evasive behaviors in windows malware," *Comput. Secur.*, vol. 113, no. C, Feb. 2022, ISSN: 0167-4048. DOI: 10.1016/j.cose.2021.102550 [Online]. Available: https://doi.org/10.1016/j.cose.2021.102550

[16] S. A. Ganugapati and S. Prabhu, "Unifying governance, risk and controls framework using sdlc, cicd and devops," in *2023 8th International Conference on Communication and Electronics Systems (ICCES)*, 2023, pp. 1797–1802. DOI: 10.1109/ICCES57224.2023.10192730

[17] M. Marandi, A. Bertia, and S. Silas, "Implementing and automating security scanning to a devsecops ci/cd pipeline," in *2023 World Conference on Communication & Computing (WCONF)*, 2023, pp. 1–6. DOI: 10.1109/WCONF58270.2023.10235015

[18] T. Abdiukov, "Automated security testing in devsecops pipelines: Integrating ai-based vulnerability discovery and compliance validation," *World Journal of Advanced Research and Reviews*, vol. 22, pp. 2083–2093, Apr. 2024. DOI: 10.30574/wjarr.2024.22.1.1083

[19] R. Mihalache, D. Gavriluț, and D. Anton, "Real-time deep learning-based malware detection using static and dynamic features," in *Proceedings of the 16th International Conference on Agents and Artificial Intelligence - Volume 3: ICAART*, INSTICC, SciTePress, 2024, pp. 226–234, ISBN: 978-989-758-680-4. DOI: 10.5220/0012316800003636

[20] T. Shi, S. Wang, H. Li, S. Guo, and Y. Liu, "Generating adversarial samples based on the attack robust training model," Jul. 2024, pp. 896–901. DOI: 10.1109/QRS-C63300.2024.00119

[21] R. Ashtagi, C. Belani, P. Bhalerao, Y. Bhalerao, and A. Chawle, "Building resilient cicd pipelines: A devops security-first framework," in *2025 International Conference on Computational, Communication and Information Technology (ICCCIT)*, 2025, pp. 828–834. DOI: 10.1109/ICCCIT62592.2025.10927871

[22] Dikshant and Verma, *Reducing false positives with active behavioral analysis for cloud security*, Aug. 2025. DOI: 10.48550/arXiv.2508.12584

[23] G. Modalavalasa and P. Yadav, "A hybrid approach to cloud database security: Integrating dl and machine learning for threat detection and prevention," in *2025 3rd International Conference on Inventive Computing and Informatics (ICICI)*, 2025, pp. 1147–1154. DOI: 10.1109/ICICI65870.2025.11069530